

IMPLEMENTATION DETAILS & SNIPPETS

Developer: [Left Blank] | OptiCrop Project Documentation

Repository: <https://github.com/goutham-05-raj/OptiCrop-Smart-Agricultural-Production-Optimization-Engine>

1. Backend Routing Code (Flask)

The controller logic utilizes Flask routes to accept POST data, process it, and render the results template. Below is an excerpt of the prediction pipeline:

- Extract soil parameters from the HTML form request.
- Transform the input list using the StandardScaler pipeline.
- Perform predictions using the Random Forest classifier.
- Predict estimated yield and categorise soil with the KMeans model.
- Append results to user_predictions.csv and render results.

Opticrop - Key Code Modules

load_all_models() [app.py]

Deserialises best_model.pkl, yield_model.pkl, scaler.pkl,

DecisionTreeClassifier.pkl, RandomForestClassifier.pkl

kmeans_model.pkl on server start into a shared dict.

predict [POST, app.py]

1. Parse & validate 7 form fields (bounds check)

2. StandardScaler transform

3. RF predict -> yield (tHa)

4. RF regress -> yield (tHa)

5. KMeans predict -> soil cluster

6. NPK delta -> fertiliser advice

7. Append row to user_predictions.csv

8. Render result Jinja2 template

6. NPK delta -> fertiliser advice

train_models.py

7. Append row to user_predictions.csv

Scales, encodes, trains 4 classifiers, selects best by accuracy,

dumps pipelines to models/ directory.

2. Model Training Script (train_models.py)

The training StandardScaler transform, splits them (80/20 train/test split), fits the StandardScaler, trains the classifiers, outputs charts, and serializes files using pickle.

train_models.py

Scales, encodes, trains 4 classifiers, selects best by accuracy,

dumps pipelines to models/ directory.